National University of Sciences & Technology (NUST)

Military College of Signals (MCS)

Department of Software Engineering

Final Year Project

Software Requirements Specification

For

Virtual OpenHouse

Application Version 1.2

Team Members:

Ayela Noor Arshad

Javeria Javed

Sabahat Farhat

Sidra Kayenat

Supervisor: Dr. Shahzaib Tahir

22th September 2025

Table of Contents

1. Introduction.....6
1.1 Purpose.....6
1.2 Document Conventions.....6
1.3 Intended Audience and Reading Suggestions.....6
1.4 Product Scope.....7
2. Overall Description.....8
2.1 Product Perspective.....8
2.2 Product Function.....10
2.2.1 User Management.....10
2.2.2 Event Management.....10
2.2.3 Stall Management.....10
2.2.4 Event Participation & Exploration.....11
2.2.5 Chatbot Assistance.....11
2.2.6 System Administration (Ultimate Admin).....11
2.2.7 Notifications & Communication.....11
2.3 User Classes and Characteristics.....12
2.4 Operating Environment.....13
2.4.1 Hardware Platform.....13
2.4.2 Operating System.....13
2.4.3 Software Environment.....13
2.4.4 Network Requirements.....14
2.5 Design and Implementation Constraints.....14
2.5.1 Technology Constraints.....14
2.5.2 Performance Constraints.....14
2.5.3 Security Constraints.....14
2.5.4 Integration Constraints.....15
2.5.5 Design Standards.....15
2.6 User Documentation.....15
2.6.2 Quick Start Guide.....15
2.6.3 Chatbot Assistance.....16
2.6.4 Delivery Formats.....16
2.7 Assumptions and Dependencies.....16
2.7.1 Technology Constraints.....16
2.7.2 Performance Constraints.....16
2.7.3 Security Constraints.....17
2.7.4 Integration Constraints.....17
2.7.5 Design Standards.....17

3. External Interface Requirements..... 17
3.1 User Interfaces..... 17
3.1.1 Overview..... 18
3.1.2 Layout and Navigation..... 18
3.1.3 Standards and Guidelines..... 19
3.1.4 Common Elements..... 19
3.1.5 Accessibility and Usability..... 20
3.1.6 Required Screens..... 20
3.2 Hardware Interfaces..... 20
3.2.1 Client Devices..... 20
3.2.2 Server Hardware..... 21
3.2.3 Input Devices..... 21
3.3 Software Interfaces..... 21
3.3.1 Operating System..... 21
3.3.2 Database..... 21
3.3.3 Web Server & Backend..... 21
3.3.4 Frontend..... 21
3.3.5 Data Exchange..... 22
3.4 Communications Interfaces..... 22
3.4.1 Protocols..... 22
3.4.2 Network Requirements..... 22
3.4.3 Security..... 22
4. System Features..... 22
4.1 Feature: User Authentication..... 23
4.1.1 Description and Priority..... 23
4.1.2 Stimulus/Response Sequences..... 23
4.1.3 Functional Requirements..... 23
4.2 Feature: Event Management (User as Event Admin)..... 24
4.2.1 Description and Priority..... 24
4.2.2 Stimulus/Response Sequences..... 24
4.2.3 Functional Requirements..... 24
4.3 Feature: Stall Allocation (User as Participant)..... 25
4.3.1 Description and Priority..... 25
4.3.2 Stimulus/Response Sequences..... 26
4.3.3 Functional Requirements..... 26
4.4 Feature: Event Attendance (User as Attendee)..... 27
4.4.1 Description and Priority..... 27
4.4.2 Stimulus/Response Sequences..... 28
4.4.3 Functional Requirements..... 28
4.5 Feature: Event Oversight (System Admin)..... 29
4.5.1 Description and Priority..... 29

4.5.2 Stimulus/Response Sequences..... 30
4.5.3 Functional Requirements..... 30
5. Other Nonfunctional Requirements..... 31
5.1 Performance Requirements.....31
5.1.1 System Response Time..... 31
5.1.2 Database Performance.....31
5.1.3 Chatbot Performance.....32
5.1.4 Reliability under Load..... 32
5.2 Safety Requirements.....32
5.2.1 Data Protection.....32
5.2.2 System Misuse Prevention.....32
5.2.3 Error Handling..... 32
5.2.4 Compliance with Policies.....33
5.3 Security Requirements.....33
5.3.1 Authentication & Authorization.....33
5.3.2 Data Privacy.....33
5.3.3 Secure Communication.....33
5.3.4 Compliance & Best Practices.....33
5.4 Software Quality Attributes.....34
5.4.1 Usability.....34
5.4.2 Reliability.....34
5.4.3 Maintainability.....34
5.4.4 Portability.....34
5.4.5 Scalability & Adaptability..... 34
5.4.6 Testability.....34
5.5 Business Rules.....35
5.5.1 Role-Based Access.....35
5.5.2 Registration & Approval.....35
5.5.3 Content Management..... 35
5.5.4 Data Integrity..... 35
5.5.5 System Usage.....35
6. Project Timeline..... 36
References.....37
Appendix A: Glossary..... 39
Appendix B: To Be Determined List..... 42

Revision History

Name	Date	Reason For Changes	Version
Javeria, Sabahat, Ayela, Sidra	26/09/2025	Adding Timeline and Diagrams, Modifying References, Adding images	1.1
Javeria, Sabahat, Ayela,Sidra	28/09/2025	Modifying Timeline and Diagrams, Modifying Section 1 and 5	1.2

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the requirements for the **Virtual OpenHouse web application (Version 1.0)**, developed as part of the Final Year Project at NUST-MCS. The document provides a detailed description of the system's functional and non-functional requirements, interfaces, and constraints.

Its purpose is to establish a common understanding among all stakeholders including the project team, supervisor, and evaluators about what the software will deliver. This SRS serves as the foundation for system design, development, testing, and future maintenance.

1.2 Document Conventions

This SRS report follows the standard format provided by the department. The document is written in Times New Roman, 12-point font size, with 1.5 line spacing and margins of 2 cm on all sides. All pages, sections, and subsections are numbered for clarity and ease of reference.

Requirement statements are written in clear, unambiguous language. Each requirement maintains its own priority, rather than inheriting from higher-level requirements. Emphasis (e.g., bold or italic text) is used only for highlighting section headings or special terms, without altering the meaning of requirements.

1.3 Intended Audience and Reading Suggestions

This document is intended for the following readers:

- **Supervisor and Evaluation Committee** – to assess the project requirements, scope, and completeness of documentation.
- **Project Team Members** – to use as a reference throughout the design, implementation, and testing phases.

- **Future Students/Researchers** – who may extend or reuse the system in future academic projects.

The document is organized in a structured manner, starting with introductory sections (Purpose, Conventions, Intended Audience), followed by detailed system requirements, functional descriptions, non-functional requirements, and supporting information.

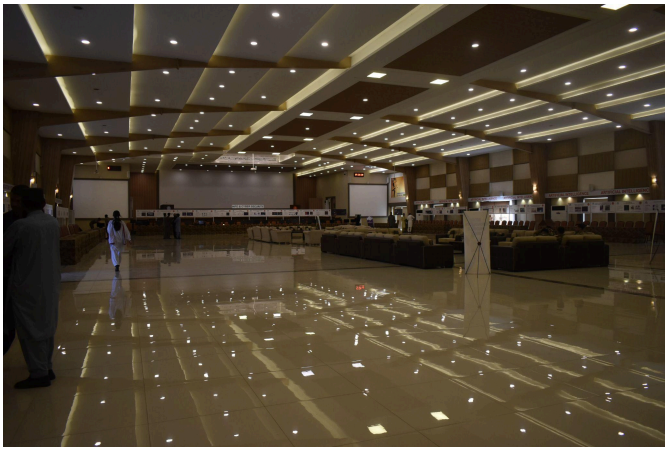
For effective use:

- **Supervisors and evaluators** may focus on the introduction, overall descriptions, and requirements sections.
- **Developers (project team)** should carefully study the detailed functional and non-functional requirements.
- **Future researchers** may review the appendices, references, and timeline to understand the system's scope and continuation potential.

Readers are encouraged to proceed sequentially from the overview sections (Section 1) to the detailed requirements (Sections 3–5), as this reflects the logical flow of the document.

1.4 Product Scope

TThe **Virtual Open House** is a web-based application designed to provide an interactive and immersive platform for hosting events such as exhibitions, showcases, and open house gatherings. Its primary purpose is to replicate the experience of a physical open house in a virtual environment, where organizers, participants, and attendees can interact seamlessly.



The system incorporates features such as role-based access, stall/project

management, an in-browser 3D event viewer, and an AI-powered chatbot for quick queries. These features aim to enhance accessibility, engagement, and information sharing across a variety of events.

The goals of this software are to:

- Digitize the concept of open house and exhibition events.
- Provide a scalable and user-friendly platform accessible from anywhere.
- Facilitate effective communication between event participants and attendees through AI assistance.
- Support adaptability for academic, professional, or community-driven events.

This system is intended as a modern solution that applies established software engineering practices to create a real-world-inspired product with practical use cases.

2. Overall Description

2.1 Product Perspective

The Virtual Tour Manager is a new, self-contained, web-based application designed to support the creation, management, and exploration of virtual events. Unlike traditional event management systems, this product integrates event organization, digital stall management, and immersive 3D environments, enabling a more engaging and interactive user experience.

This product is not a follow-on member of an existing product family, nor is it a replacement for current systems. Instead, it enhances the digital event experience by combining event management, interactive content delivery, and AI-powered support within a unified system.

The product will operate as a standalone web application, accessible through modern browsers. To achieve its functionality, it integrates with external technologies, including:

- **Three.js** for rendering immersive 3D environments in web-based applications (e.g., predefined halls and the custom university hall).
- **Database and storage services** for maintaining user accounts, event details, stall information, and uploaded multimedia resources.
- **AI-powered chatbot** that uses stall-provided data to answer user queries and improve attendee interaction.

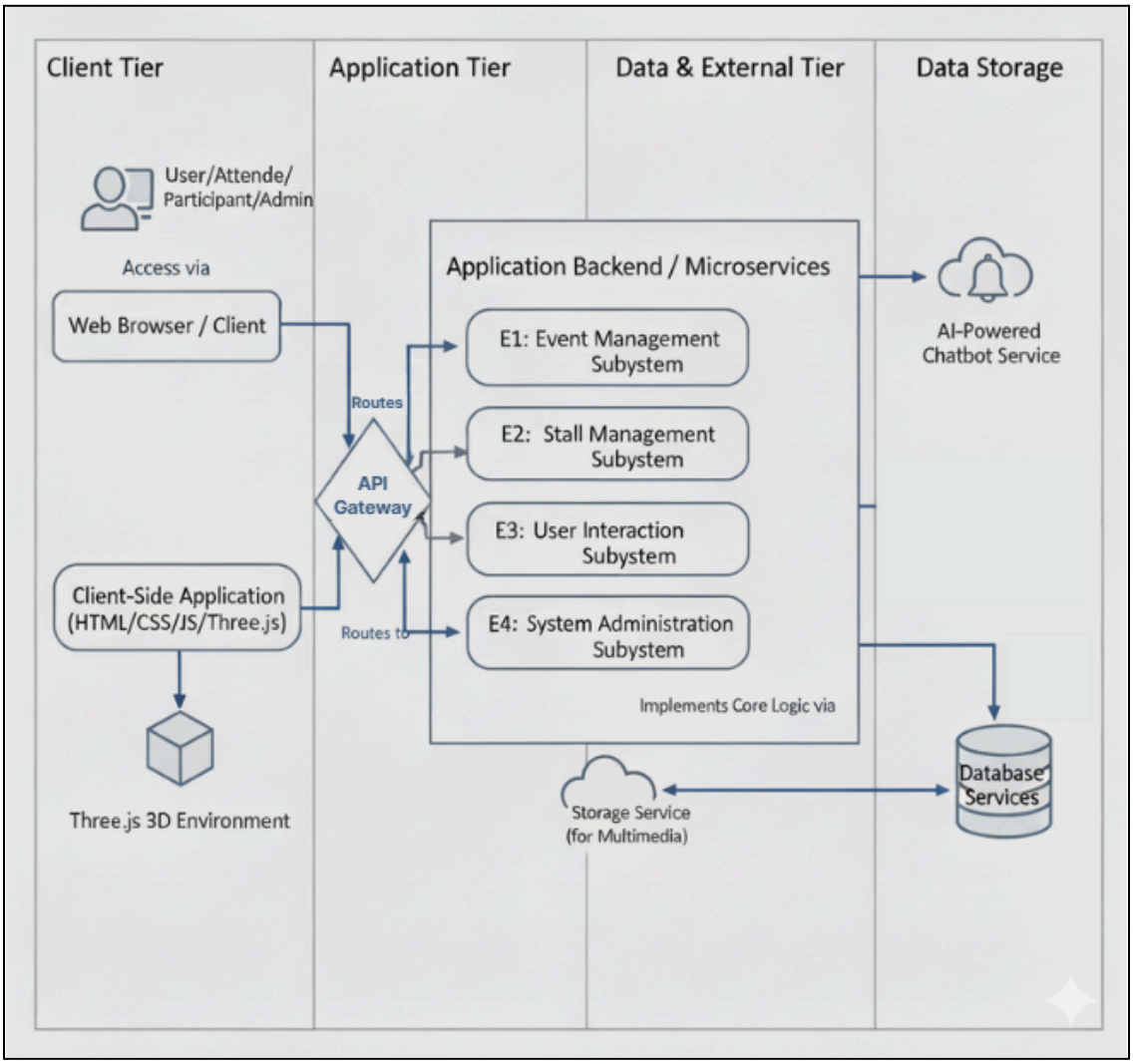


Figure 1: System Architecture Diagram

The system architecture will consist of the following major subsystems, as shown in figure 1:

1. **Event Management Subsystem** – Supports event CRUD operations, environment selection, and approval of stall registration requests.
2. **Stall Management Subsystem** – Enables registered users to request stalls, upload stall-specific multimedia content, and engage with event visitors.
3. **User Interaction Subsystem** – Provides features for event browsing, stall exploration, and chatbot-based assistance.
4. **System Administration Subsystem** – Provides dashboards for monitoring platform-wide activity, and ensuring secure and appropriate content usage.

2.2 Product Function

2.2.1 User Management

- Register, authenticate, and manage user accounts.
- Assign user roles: System Admin, Event Admin, Participant, Attendee.
- Enforce role-based access control.

2.2.2 Event Management

- Create, update, view and delete events.
- Allow Event Admins to configure event details (title, description, schedule, environment).
- Provide dashboards for event monitoring.

2.2.3 Stall Management

- Enable Participants to request stalls in events.
- Allow Event Admins to approve/reject stall requests.
- Support uploading and updating stall content (text, documents, images,

videos, external links).

- Display stalls interactively within the 3D environment.

2.2.4 Event Participation & Exploration

- Allow Attendees to browse Live and Upcoming events.
- Enable users to register for upcoming events.
- Provide a 3D immersive environment (via Three.js) where attendees can explore stalls, view content, and interact with exhibitors.

2.2.5 Chatbot Assistance

- Provide an AI-powered chatbot trained on stall/event data.
- Allow attendees to ask questions and retrieve information about stalls, schedules, or event content.

2.2.6 System Administration (Ultimate Admin)

- Manage users.
- Monitor events and stall activity.
- Oversee system-wide configurations and ensure compliance with usage policies.

2.2.7 Notifications & Communication

- Send notifications (event registration, stall approval, account creation).

2.3 User Classes and Characteristics

User Class	Responsibilities	Frequency of Use	Privilege Level	Technical Expertise	Importance
System Admin	Manage users, oversee events, configure system settings, ensure compliance.	Moderate (Monitoring etc.,).	Full access	Intermediate (Dashboard usage).	High – ensures stability/security.
Event Admin	Create/manage events, approve stalls, manage participants and event settings.	High (Especially before/during events).	Event-level admin	Basic–Intermediate.	High – central to event creation.
Participant	Register stalls, upload stall content (docs, videos, links).	Medium–High (before events).	Limited (own stalls only)	Basic (content upload/navigation).	Medium–High – provides stall content.
Attendee	Browse events, explore stalls in a 3D environment,	High (mostly during live events).	Read-only (view & interact)	Minimal (basic browsing).	High – primary audience

	and interact with chatbot.				
Chatbot (System Role)	Answer queries, provide stall/event info.	Continuous (always available).	System integration only	N/A (automated).	Medium – improves usability.

2.4 Operating Environment

2.4.1 Hardware Platform

- **Server-side (Hosting Environment):**
  - Cloud server (e.g., AWS, Azure, or any VPS).
  - Minimum: 2 vCPUs, 4 GB RAM, 50 GB storage.
  - Scalable infrastructure to support concurrent users.
- **Client-side (User Devices):**
  - Desktop/laptop with modern web browser.
  - Smartphones/tablets (Android ≥ 9, iOS ≥ 13).

2.4.2 Operating System

- **Server:** Linux (Ubuntu 20.04+), Windows Server 2019+ (depending on deployment).
- **Client:** Cross-platform (Windows 10+, macOS 11+, Android 9+, iOS 13+).

2.4.3 Software Environment

- **Backend:** Node.js / Express.js (REST API), Python.
- **Frontend:** React.js (or Next.js for SSR).
- **Database:** Non-Relational / Relational.
- **Chatbot:** NLP/AI service integration (Dialogflow or custom-trained ML model)

- **Web Browser Support:** Chrome (latest), Firefox (latest), Edge, Safari.

2.4.4 Network Requirements

- Reliable internet connection with minimum 20 Mbps per user.
- Secure HTTPS protocol for all communications.

2.5 Design and Implementation Constraints

2.5.1 Technology Constraints

- The system must be developed as a web-based application accessible via modern browsers.
- The 3D environment must use a WebGL-supported framework (e.g., Three.js or equivalent).
- The project depends on the feasibility of the selected 3D environment creation method (panoramas, HDR techniques, or 3D modeling with Blender). The choice of method may affect development time, rendering performance, and user experience.
- A database (relational or non-relational) will be required to manage users, events, and stalls.
- A cloud storage service (e.g., AWS S3, Firebase, Google Cloud Storage, Cloudinary) will be required to store user-uploaded attachments such as documents, images, and videos.

2.5.2 Performance Constraints

- The system should support multiple concurrent users without performance degradation.

2.5.3 Security Constraints

- All communication must be conducted over secure channels (HTTPS/TLS).
- Role-based access control (RBAC) must be implemented to distinguish between

different user roles (admin, event admin, participant, attendee).

2.5.4 Integration Constraints

- The system must support integration with an email/notification service for user communication.
- External APIs (e.g., authentication, storage, or chatbot) may be used as needed.
- The chatbot must be trained on stall-provided content (text, documents, FAQs), starting as retrieval-based .

2.5.5 Design Standards

- The user interface must follow responsive design principles to support desktop and mobile devices.
- Accessibility standards (WCAG 2.1 Level AA) should be followed where feasible.

2.6 User Documentation

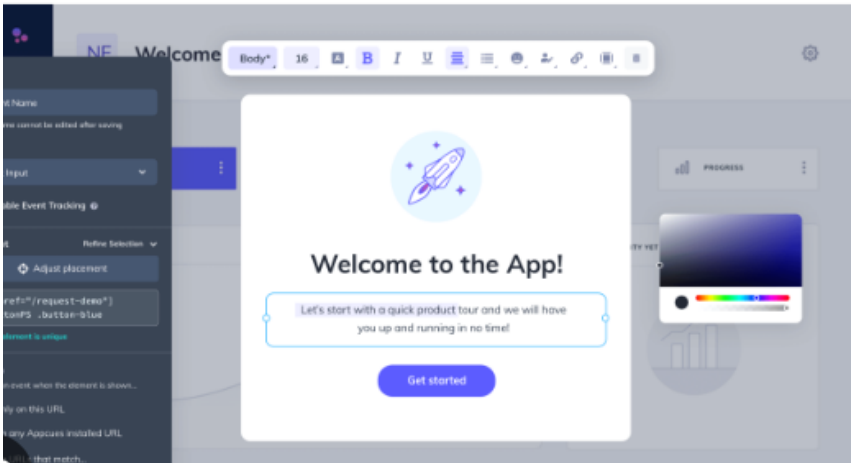
The following user documentation will be provided to ensure smooth onboarding and effective usage of the Virtual Tour Manager system:

2.6.1 User Manual (PDF/Online Guide)

Comprehensive instructions for all user roles (System Admin, Event Admin, Participant, Attendee). Covers system navigation, event creation, stall setup, and 3D environment exploration.

2.6.2 Quick Start Guide

Concise step-by-step instructions to help new users quickly get



started. Includes visual screenshots for basic tasks such as login, event registration, and stall setup.

2.6.3 Chatbot Assistance

Integrated chatbot capable of answering user questions regarding stall data and basic system usage. Provides real-time guidance to attendees and participants.

2.6.4 Delivery Formats

PDF format for downloadable manuals.

2.7 Assumptions and Dependencies

2.7.1 Technology Constraints

The system must be developed as a web-based application accessible via modern browsers.The 3D environment must use a WebGL-supported framework (e.g., Three.js or equivalent). A database (relational or non-relational) will be required to manage users, events, and stalls. A cloud storage service (e.g., AWS S3, Firebase, Google Cloud Storage) will be required to store user-uploaded attachments such as documents, images, and videos. The virtual university hall environment should preferably be created using HDRs, panoramas, or image stitching techniques to provide a lightweight and immersive experience. If these methods do not meet the quality or interactivity requirements, a custom 3D model of the hall will be developed using tools such as Blender, optimized for WebGL rendering.

2.7.2 Performance Constraints

The system should support multiple concurrent users without performance degradation. Client devices must support basic WebGL rendering to access the 3D

event environment.

2.7.3 Security Constraints

All communication must be conducted over secure channels (HTTPS/TLS). Role-based access control (RBAC) must be implemented to distinguish between different user roles (admin, event admin, participant, attendee).

2.7.4 Integration Constraints

The system must support integration with an email/notification service for user communication. External APIs (e.g., authentication, storage, or chatbot) may be used as needed. The chatbot must be trained on stall-provided content (text, documents, FAQs). Initially, a retrieval-based model will be used; later, AI/ML frameworks or external APIs (e.g., OpenAI, Dialogflow, Rasa) may be integrated for more advanced conversational capabilities.

2.7.5 Design Standards

The user interface must follow responsive design principles to support desktop and mobile devices. Accessibility standards (WCAG 2.1 Level AA) should be followed where feasible.

3. External Interface Requirements

3.1 User Interfaces

The system will provide a web-based graphical user interface (GUI) accessible from modern browsers (e.g., Chrome, Edge, Firefox). The interface will maintain a professional, consistent, and intuitive design across all screens.

3.1.1 Overview

The application supports multiple user roles with distinct access levels:

- **System Admin** – manages the entire platform, including users, events, and system configurations.
- **Event Admin** – creates and manages events, stalls, and participant allocations.
- **Participant** – registers stalls and uploads details (project descriptions, media, team info).
- **Attendee** – accesses events, views schedules, and explores stall content in a 3D virtual environment powered by Three.js.

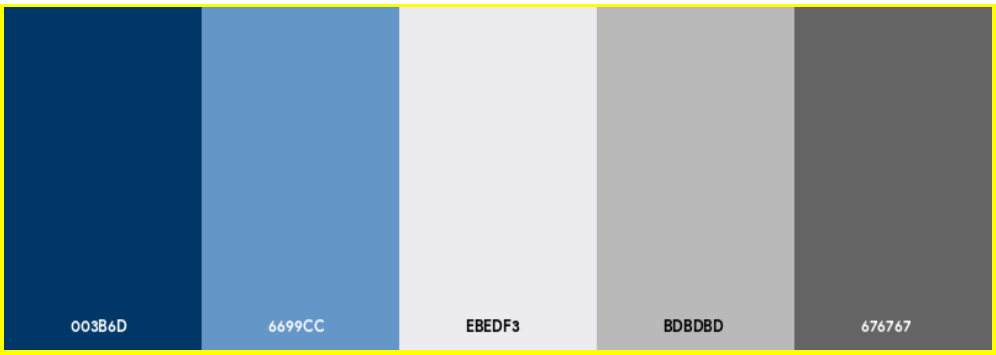
3.1.2 Layout and Navigation

- The **homepage** displays event categories: *Live Events, Upcoming Events, Archive Events*.
- A **navigation bar and/or sidebar** provide quick access, adapted per role:
  - **System Admin** – user management, event management, system settings.
  - **Event Admin** – event dashboard, stall assignment, participant management.
  - **Participant** – stall registration, profile management, submission updates.
  - **Attendee** – event browsing, 3D stall exploration, and schedule viewing.

- The **3D event space** will load within the browser, allowing attendees to "walk" and interact with stalls directly.

3.1.3 Standards and Guidelines

- **Material Design principles** will guide the UI.
- Color palette: **blue, white, grey**.
- Typography: clean, sans-serif fonts (e.g., Roboto, Open Sans).
- Button standards:
  - Primary → blue highlight.
  - Secondary → grey.
- Icons will use a **minimal, flat style**.



3.1.4 Common Elements

- Navigation bar/sidebar for role-based access.
- **Logout** option on every page.
- **Help/Support** accessible globally.

- **Error handling:** inline field validation for minor issues, modal dialogs for critical errors.

3.1.5 Accessibility and Usability

The system will ensure accessibility and usability by complying with WCAG 2.1 Level AA standards, supporting screen readers, providing alternative text for images, using high-contrast colors, and maintaining a responsive design for desktop, tablet, and mobile devices.

3.1.6 Required Screens

- Login/Registration – all users.
- Dashboard – role-specific.
- Create Event – Event Admins.
- Profile – personal/organizational info.
- Settings – account/system preferences.
- 3D Event Viewer (Three.js) – for Attendees to explore stalls interactively.

3.2 Hardware Interfaces

The system is **web-based** and requires no specialized hardware.

3.2.1 Client Devices

- Desktops/laptops with modern browsers.
- Mobile devices (responsive design supported).

3.2.2 Server Hardware

- Web server capable of scaling (cloud or VM).
- Adequate CPU, memory, and storage for concurrent users.

3.2.3 Input Devices

- Standard (keyboard, mouse, touchscreen).

3.3 Software Interfaces

3.3.1 Operating System

The system will run on a Windows Server or an equivalent Linux-based server, while clients can access it from any operating system that supports modern web browsers.

3.3.2 Database

The system will utilize either a relational or non-relational database, depending on scalability and performance needs. The database will store structured and semi-structured data including user profiles, event details, stall information, schedules, and chatbot interactions

3.3.3 Web Server & Backend

- Node.js with Express.js framework.

3.3.4 Frontend

- React.js for dynamic, responsive UI.
- Three.js for rendering the 3D virtual event space.

3.3.5 Data Exchange

Data exchange will be handled through JSON-based requests and responses, with RESTful API endpoints facilitating communication. Authentication will be implemented using JWT or OAuth 2.0 to ensure secure access.

3.4 Communications Interfaces

3.4.1 Protocols

- HTTP/HTTPS for client-server interactions.
- TLS 1.2+ for encrypted communication.

3.4.2 Network Requirements

- Stable internet connection (≥ 20 Mbps recommended).
- Optimized for broadband and mobile data.

3.4.3 Security

The system will ensure security through encrypted sessions using **SSL/TLS**, **role-based access control**, and secure session management with anti-hijacking measures.

4. System Features

This section describes the major functional features of the system, organized by the roles and workflows it supports. Each feature corresponds to a core service provided by the platform, ensuring that users, including Admins, Event Admins, Participants, and Attendees, can effectively interact with and benefit from the system. For each feature, the description, priority, stimulus/response sequences, and functional requirements are outlined to define the expected behavior and scope of the product.

4.1 Feature: User Authentication

This feature manages secure login and role-based access, ensuring users can only access functionality appropriate to their role (Admin or User).

4.1.1 Description and Priority

The User Authentication feature ensures that only authorized users can access the system by providing secure login functionality for both Admins and Users. It distinguishes between different user roles and grants them access to their respective dashboards. Since authentication is the entry point to the platform and a prerequisite for all other workflows, this feature is of **High priority**, with maximum benefit and penalty ratings, and a relatively low cost and risk of implementation.

4.1.2 Stimulus/Response Sequences

- **Stimulus**: User enters login credentials.
  Response: System verifies credentials and grants access to Admin or User dashboard.

- **Stimulus**: User enters invalid credentials.
  Response: System displays error message and denies access.

4.1.3 Functional Requirements

- **REQ-1**: The system shall provide a login interface for users to enter credentials.

- **REQ-2**: The system shall validate credentials before granting access.

- **REQ-3**: The system shall redirect valid Admin logins to the Admin homepage and valid User logins to the User homepage.

- **REQ-4**: The system shall display appropriate error messages for invalid login attempts.

4.2 Feature: Event Management (User as Event Admin)

This feature enables Event Admins to create and configure events by defining details such as description, schedule, stalls, and access type, as shown in figure 2.

4.2.1 Description and Priority

The Event Management feature allows Users acting as Event Admins to create, configure, and manage events. Event details include name, description, date and time, and number of stalls. This feature forms the foundation of the system since it enables the creation of the virtual events themselves, making it a **High priority** requirement. The benefits of this feature are significant in terms of system usability, while the cost and risks are moderate.

4.2.2 Stimulus/Response Sequences

- **Stimulus**: Event Admin selects "Create Event."
  Response: System prompts for event details (name, description, date/time, stalls, access type).

- **Stimulus**: Event Admin submits event details.
  Response: System saves and displays events under Upcoming Events.

4.2.3 Functional Requirements

- **REQ-1**: The system shall allow Event Admins to create events with details (name, description, date/time, stalls).

- **REQ-2**: The system shall allow Event Admins to edit or delete created events.

- **REQ-3**: The system shall display events under Live or Upcoming categories.
- **REQ-4**: The system shall handle invalid inputs (e.g., past dates) by displaying error messages

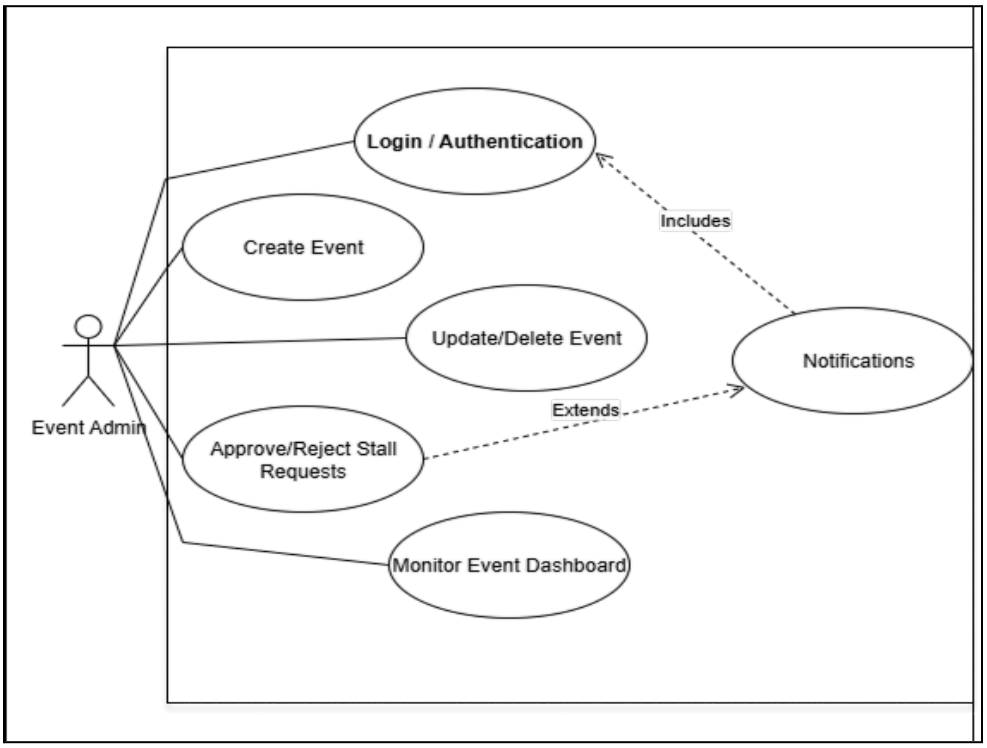


Figure 2: Use Case Diagram of Event Admin

4.3 Feature: Stall Allocation (User as Participant)

This feature supports participants by allowing them to request stalls, get approvals, and publish project details in their allocated space, as shown in figure 3.

4.3.1 Description and Priority

The Stall Allocation feature enables Users acting as Participants to request stalls in upcoming events, receive approval or rejection from Event Admins, and submit project details for their assigned stalls. This feature directly supports the core

objective of showcasing student or participant projects in virtual events. Its importance to the event experience makes it a **High priority**, with strong benefits and penalties, while the implementation risk remains moderate.

4.3.2 Stimulus/Response Sequences

- **Stimulus**: Participant sends a request for a stall.
  Response: System notifies Event Admin for approval/rejection.

- **Stimulus**: Event Admin approves request.
  Response: System assigns stall to Participant.

- **Stimulus**: Participant submits project details.
  Response: System displays project details in assigned stall during the event.

4.3.3 Functional Requirements

- **REQ-1**: The system shall allow Participants to request stalls for upcoming events.

- **REQ-2**: The system shall notify Event Admins of stall requests.

- **REQ-3**: The system shall enable Event Admins to approve/reject stall requests.

- **REQ-4**: The system shall allow Participants to submit project details once stalls are assigned.

- **REQ-5**: The system shall display the submitted project details in the respective stall during the event.

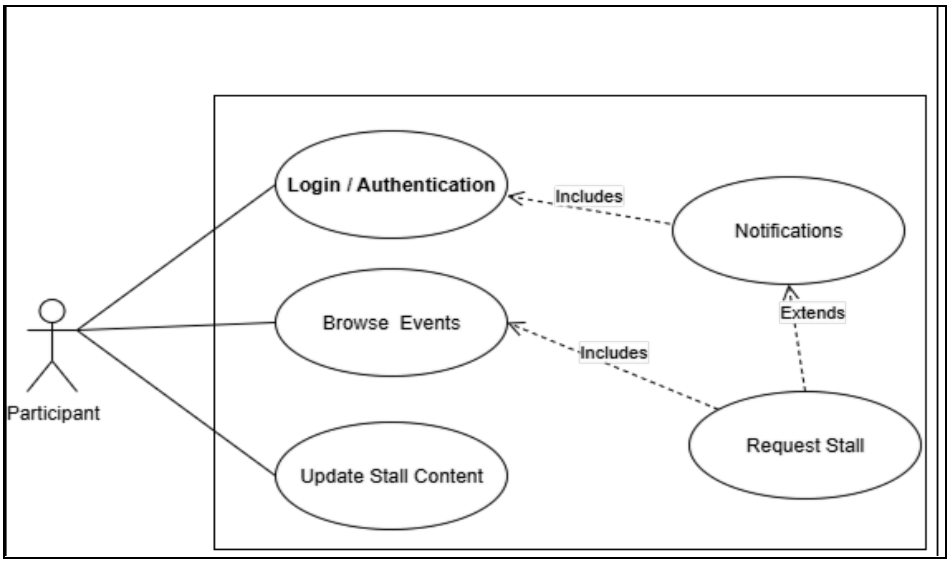


Figure 3: Use Case Diagram of Participant

4.4 Feature: Event Attendance (User as Attendee)

This feature provides attendees with the ability to explore events, visit stalls, and interact with project details through navigation and chatbot support, as shown in figure 4.

4.4.1 Description and Priority

The Event Attendance feature allows Users acting as Attendees to join live or upcoming events, navigate through stalls, and interact with project and team information using an integrated chatbot. As this feature ensures the usability of the platform for its primary audience, the event visitors, it is a **High priority** feature. The benefits are high due to improved user experience, while the cost and risks associated with its development are relatively low.

4.4.2 Stimulus/Response Sequences

- **Stimulus**: Attendee selects a live event.
  Response: System verifies access rights.

- **Stimulus**: Attendee navigates stalls.
  Response: System displays project title and team members.

- **Stimulus**: Attendee queries chatbot.
  Response: Chatbot responds with project/team details.

4.4.3 Functional Requirements

- **REQ-1**: The system shall allow Attendees to access live/open events without restrictions.

- **REQ-2**: The system shall allow Attendees to navigate between stalls.

- **REQ-3**: The system shall display stall information (project title, team names).

- **REQ-4**: The system shall provide a chatbot for project queries.

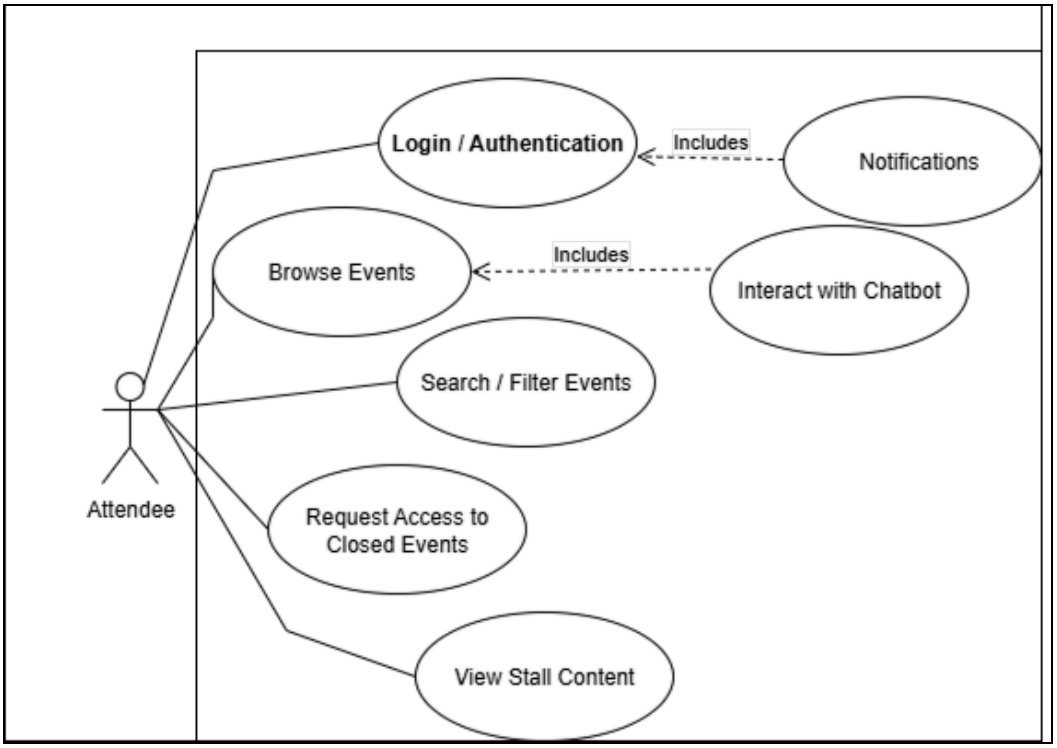


Figure 4: Use Case Diagram of Attendee

4.5 Feature: Event Oversight (System Admin)

This feature equips system administrators with oversight capabilities to manage events, enforce rules, and ensure smooth operations, as shown in figure 5.

4.5.1 Description and Priority

The Event Oversight feature gives Admins complete control over the system, including monitoring events, suspending or canceling them, and managing users. It also ensures platform stability and fairness by enforcing policies. Given that this feature underpins governance and trust in the platform, it is of **High priority**, with very high benefits and penalties in case of failure, although implementation cost and risks are somewhat higher compared to other features.

4.5.2 Stimulus/Response Sequences

- **Stimulus**: Admin logs into dashboard.
  Response: System displays all events with status (Live, Upcoming).
- **Stimulus**: Admin suspends/cancels event.
  Response: System updates event status and notifies Event Admin.
- **Stimulus**: Admin removes users.
  Response: System revokes access for that user.
- **Stimulus**: Admin approves event creation request.
  Response: System notifies the event admin about the request approval.

4.5.3 Functional Requirements

- **REQ-1**: The system shall allow Admins to view all events and statuses.
- **REQ-2**: The system shall allow Admins to approve, suspend or cancel events.
- **REQ-3**: The system shall allow Admins to manage users.
- **REQ-4**: The system shall log Admin actions for auditing.
- **REQ-5**: The system shall handle errors gracefully.

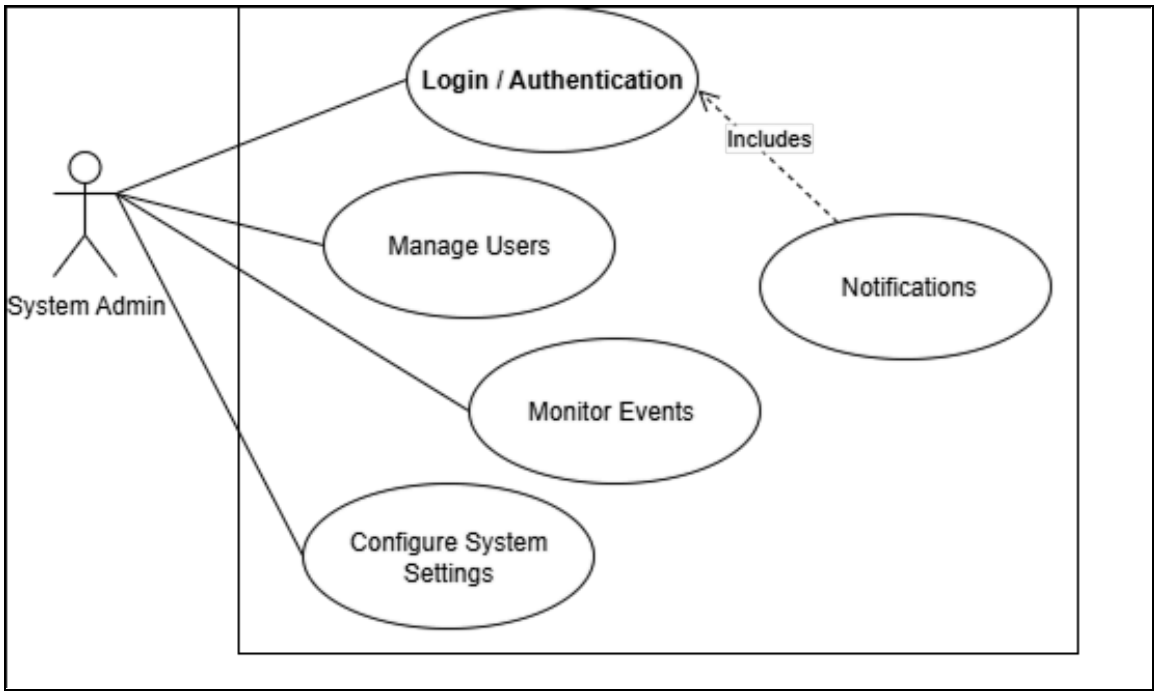


Figure 5: Use Case Diagram of System Admin

5. Other Nonfunctional Requirements

5.1 Performance Requirements

5.1.1 System Response Time

- The application shall load the dashboard and main pages within an acceptable time frame to ensure smooth user interaction.
- The 3D Event Viewer shall initialize and render efficiently for events of varying sizes, maintaining a seamless experience for attendees.

5.1.2 Database Performance

- Queries related to events, stalls, and user data shall return results promptly under normal operating conditions.
- The system shall be able to manage a reasonable number of records without noticeable performance issues.

5.1.3 Chatbot Performance

- The integrated AI chatbot shall respond to user queries in a timely manner, providing information without causing noticeable delays.

5.1.4 Reliability under Load

- The system shall remain stable and functional during active event sessions, even under moderate to high usage.
- In case of high demand, the system shall degrade gracefully (e.g., slower responses rather than errors or crashes)..

Rationale: These requirements ensure that the Virtual Open House delivers a smooth, interactive, and reliable experience to attendees, stall owners, and administrators across different usage conditions.

5.2 Safety Requirements

5.2.1 Data Protection

- The system shall ensure that user data (login credentials, personal details, and project information) is securely stored and not exposed to unauthorized access.
- Regular backups of the database shall be maintained to prevent data loss in case of system failure.

5.2.2 System Misuse Prevention

- Unauthorized modifications to stalls, events, or user information shall be prevented through role-based access control.

5.2.3 Error Handling

- In the event of a failure (e.g., 3D viewer crash or server downtime), the system shall provide clear error messages and safe recovery options without data corruption.

5.2.4 Compliance with Policies

- The system shall follow general web application security and data handling best practices.
- It shall not collect or misuse sensitive personal data beyond its intended purpose.

Rationale: These safety measures ensure that the Virtual OpenHouse platform avoids harm in the form of data loss, unauthorized changes, or system misuse, which could negatively affect participants, administrators, and the credibility of the event.

5.3 Security Requirements

5.3.1 Authentication & Authorization

- The system shall require all users (admins, stall owners, attendees) to authenticate using a valid username and password before accessing system features.
- Role-based access control shall be enforced to ensure users can only access features relevant to their role.

5.3.2 Data Privacy

- User credentials shall be stored in encrypted form.
- Personal data of users and event-related information shall not be shared with unauthorized parties.

5.3.3 Secure Communication

- All data transmission between client and server shall use secure protocols to prevent interception.
- Sensitive operations (e.g., login, stall registration, updates) shall be safeguarded against unauthorized interference.

5.3.4 Compliance & Best Practices

- The system shall align with widely accepted security guidelines (e.g., OWASP Top 10).
- It shall avoid unnecessary collection of sensitive data beyond its operational scope.

Rationale: These security requirements protect the integrity of the Virtual OpenHouse platform,

ensure user privacy, and safeguard event data against unauthorized access.

5.4 Software Quality Attributes

5.4.1 Usability

- The system shall provide a clean and intuitive interface, enabling users to navigate the platform easily without prior training.
- The 3D Event Viewer shall offer simple, intuitive controls for exploring stalls.

5.4.2 Reliability

- The platform shall remain consistently available during events, with critical functions continuing to operate under typical usage conditions.

5.4.3 Maintainability

- The system shall be developed using a modular architecture to support updates and extensions with minimal effort.
- Code shall be documented to facilitate future modifications or enhancements.

5.4.4 Portability

- The application shall be accessible via standard web browsers without requiring additional installations.
- It shall operate across commonly used operating systems without compatibility issues.

5.4.5 Scalability & Adaptability

- The platform shall support increasing numbers of stalls and attendees as events expand, without requiring fundamental redesign..

5.4.6 Testability

- All functional requirements shall be testable against defined acceptance criteria.
- Automated testing (unit and integration) shall be incorporated where possible to ensure correctness.

Rationale: These attributes ensure that the Virtual OpenHouse is not only functional but also user-friendly, reliable, and extendable, meeting both the immediate academic goals and potential future needs.

5.5 Business Rules

5.5.1 Role-Based Access

- Only Event Admins can create, update, or delete events.
- Only Stall Owners can register and manage their stalls/projects.
- **Attendees** can browse stalls, interact with the 3D Event Viewer, and use the chatbot, but cannot modify content.

5.5.2 Registration & Approval

- Stall registration requests must be reviewed and approved by an Event Admin before becoming visible in the event.
- Each stall must be associated with exactly one registered stall owner.

5.5.3 Content Management

- Stall owners may upload project details (text, images, videos) but must comply with event guidelines (e.g., no offensive or unrelated content).
- Event Admins reserve the right to remove stall content if it violates policies.

5.5.4 Data Integrity

- No user may modify or delete another user's data (e.g., stall details, account information) without appropriate permissions.
- All critical updates (stall edits, deletions, event updates) shall be logged for accountability.

5.5.5 System Usage

- Users must access the system with valid login credentials; anonymous access is restricted to viewing homepage only.

Rationale: These rules ensure fair usage of the system, protect data integrity, and enforce clear responsibilities among different roles.

6. Project Timeline

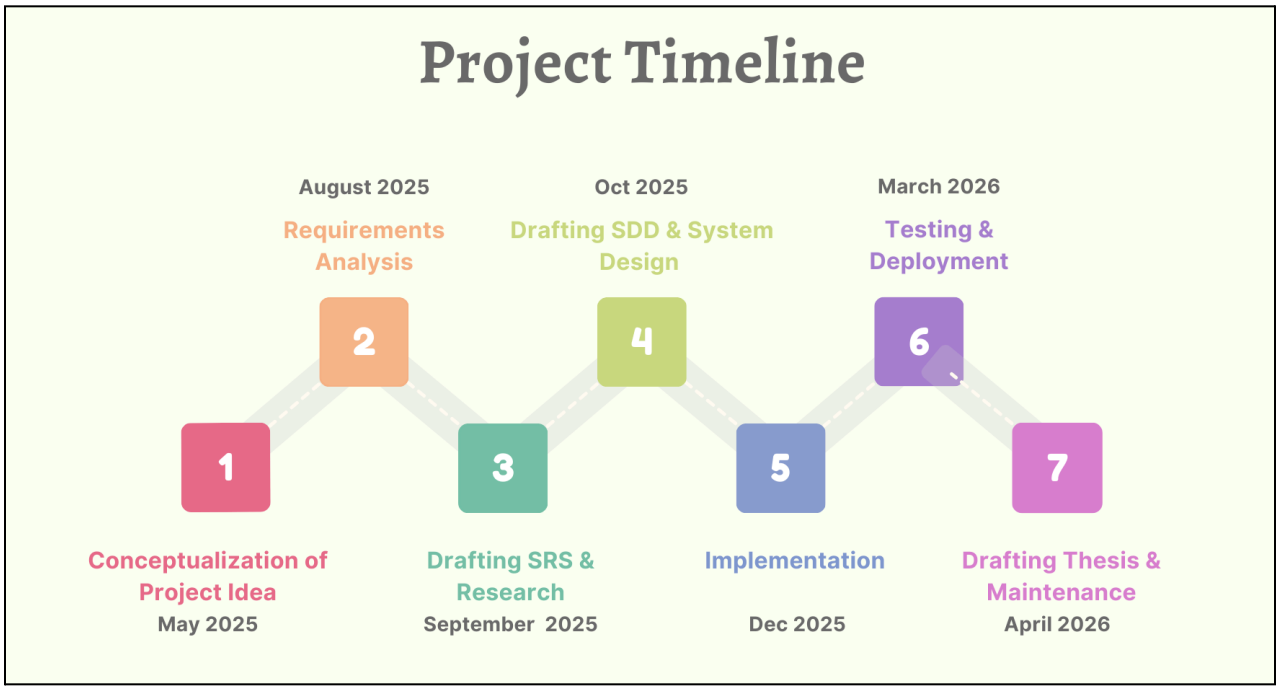


Figure 6: Project Timeline

References

[1] IEEE Computer Society, *IEEE Recommended Practice for Software Requirements Specifications*, IEEE Std 830-1998, 1998.

[2] Department of Software Engineering, NUST-MCS, *Final Year Project SRS Guidelines and Template*, 2025.

[3] React Team, "React — Official Documentation." [Online]. Available: https://react.dev

[4] Three.js Contributors, "Three.js — Official Documentation." [Online]. Available: https://threejs.org/docs

[5] vFairs, "Event Management Platform." [Online]. Available: https://www.vfairs.com/event-management-platform/

[6] V21 Artspace, "Virtual Exhibitions." [Online]. Available: https://v21artspace.com/

[7] K. F. Eng, "So you want to make a virtual exhibition," *Medium*, Aug. 2020. [Online]. Available: https://karenfranceseng.medium.com/so-you-want-to-make-a-virtual-exhibition-4a002f43e6e7

[8] A. M. Rahaman, M. A. Islam, and T. S. R. Begum, "3D Virtual Exhibitions," *ResearchGate*, Sep. 2013. [Online]. Available: https://www.researchgate.net/publication/257137680_3D_Virtual_Exhibitions

[9] MootUp, "Virtual Event Platform." [Online]. Available: https://mootup.com/

[10] Brain Station 23, "Creating a 3D virtual expo for WebGL with Unity 3D," *Medium*, Apr. 2019. [Online]. Available: https://medium.com/brainstation23/creating-a-3d-virtual-expo-for-webgl-with-unity-3d-853a4b6a7ebb

[11] Artsteps, "Virtual Exhibition Platform." [Online]. Available: https://www.artsteps.com/

[12] vFairs, “Virtual Exhibitions Guide.” [Online]. Available: https://www.vfairs.com/guides/virtual-exhibitions/

[13] R. I. Barraza Castillo, A. L. De La Torre Rodríguez, I. I. Méndez-Gurrola, and A. Solís Chávez, “Design of a Bespoke Web-Based 3D Virtual Venue and Video Streaming Event Platform,” in *Creating Digitally*, A. L. Brooks, Ed., Intelligent Systems Reference Library, vol. 241, Cham: Springer, 2023, pp. 527-547. doi: 10.1007/978-3-031-31360-8_21.

[14] L. Li, “The Design and Implementation of Digital Virtual Exhibition Platform for Arts and Crafts Design Based on 3D Web3D Technology,” in *Proc. 2023 2nd Int. Conf. Educ. Innov. Multimedia Technol. (EIMT)*, Atlantis Press, 2023, pp. 87–91. doi: 10.2991/978-94-6463-192-0_12. Available: https://www.atlantis-press.com/proceedings/eimt-23/125988591

Appendix A: Glossary

3D Event Viewer

An interactive, browser-based 3D interface (developed using Three.js) that allows attendees to navigate stalls virtually.

Admin

A system-level user with the highest authority. Responsible for overseeing events, managing users, and enforcing platform policies, including suspending or canceling events.

Attendee

A type of User who joins and explores live or upcoming events. Attendees can navigate stalls, view project information, and interact with a chatbot to obtain further details.

Chatbot

An integrated conversational interface available to attendees within events. It provides additional information about projects, teams, or event details.

Dashboard/Home

The main landing page displayed after login, tailored according to the user role (Admin or User). It shows live events, upcoming events, and event management options.

Event

A virtual exhibition or gathering within the system, created by an Event Admin. Events contain stalls where participants can showcase their projects. Events can be categorized as live or upcoming.

Event Admin

A User who creates and manages events. Event Admins can define event details, allocate stalls, and approve or reject participant requests.

Event Attendance

The process by which a User (as Attendee) joins an event, explores stalls, and interacts with available features.

Event Management

The set of functions performed by an Event Admin, including event creation, editing, deletion, and configuration of stalls and access type.

Live Event

An event that is currently active and accessible to attendees in real time.

Open Event

An event type that is accessible to all users without restriction.

Participant

A User who requests stalls within upcoming events, submits project details once approved, and showcases work during live events.

Stall

A virtual booth allocated to a participant in an event. Each stall contains project details and serves as the interaction point for attendees.

Stall Allocation

The process of requesting, approving, and assigning stalls to participants within an event.

System Oversight

Administrative functions carried out by the Admin role to ensure smooth operation of the platform, including monitoring events and user activity.

User

A general system role that can act as an Event Admin, Participant, or Attendee depending on the selected workflow.

Appendix B: To Be Determined List

The following is a numbered list of TBD references that require further specification or clarification in the SRS:

TBD: Maximum number of concurrent users the system can support is yet to be determined.

TBD: The scope and behavior of archived events are not yet finalized. It is to be determined whether archived events will be accessible only to Admins, Event Admins, or all users. The workflow for archiving (manual by Admin, automatic after event ends, or event-admin controlled) and the retention policies for archived event data (e.g., project details, chat history) remain to be defined.

TBD: The decision regarding the type of database to be used is not yet finalized. It is to be determined whether the system will use a relational database (e.g., MySQL, PostgreSQL) for structured event, user, and stall data, or a non-relational database (e.g., MongoDB, Firebase) for flexible and scalable data handling. The choice will depend on performance requirements, data structure complexity, and integration needs.

TBD: The method for creating and integrating custom backgrounds (e.g., MCS Open House) is yet to be finalized. Options include using photo stitching/panoramas, HDR techniques, or detailed 3D modeling in tools like Blender, depending on feasibility, performance constraints, and desired level of immersion.